Installer MongoDb et node

Bonjour à tous ! Ravi de vous accompagner. Aujourd'hui, nous allons décomposer ensemble la création d'une application **CRUD** (Create, Read, Update, Delete). C'est le "Hello World" des développeurs Full-Stack : une application capable de créer des données, de les lire, de les modifier et de les supprimer.

Imaginez que nous construisons une petite armoire de rangement numérique.

Voici les étapes, pas à pas.

Étape 1 : Préparer le terrain (L'environnement)

Avant de coder, il faut les bons outils.

1. **Node.js** : C'est notre moteur. Il permet d'exécuter du JavaScript sur notre ordinateur (et non plus seulement dans le navigateur).
2. **MongoDB** : C'est notre entrepôt de données. Contrairement à Excel, il range les données sous forme de "documents" (qui ressemblent à des objets JavaScript).
3. **npm (Node Package Manager)** : C'est notre catalogue d'outils. On va y piocher trois outils indispensables :
    - **Express** : Pour créer le serveur web.
    - **Mongoose** : C'est le traducteur entre notre code et la base de données.
    - **Dotenv** : Pour cacher nos secrets (mots de passe, ports) dans un fichier `.env`.

Étape 2 : Le fichier `.env` (Le coffre-fort)

C'est ici qu'on définit les variables de configuration.

- **Pourquoi ?** Pour que votre code puisse fonctionner sur n'importe quel ordinateur sans avoir à changer le code source.
- **Attention** : Pas de code JavaScript ici, juste des noms et des valeurs.

Étape 3 : Le fichier `server.js` (Le cerveau)

C'est le fichier central. On y fait trois choses majeures :

1. **La Connexion** : On dit à Node de se connecter à MongoDB. Si la connexion échoue, on ne lance pas le serveur (sécurité d'abord !).
2. **Le Modèle (Schema)** : C'est le "moule". On définit que chaque "Utilisateur" doit avoir un nom, un email et un âge. Mongoose surveillera que personne n'essaie d'enregistrer n'importe quoi.

3. **Les Routes (API)** : Ce sont les adresses que le navigateur va appeler.
    - `POST /users` -> "Tiens, crée cet utilisateur."
    - `GET /users` -> "Donne-moi la liste."
    - `PUT /users/:id` -> "Remplace les infos de celui qui a cet ID."
    - `DELETE /users/:id` -> "Supprime celui-là."

Étape 4 : Le dossier `public` (L'interface utilisateur)

Node.js est "invisible". Pour que l'utilisateur puisse interagir, on crée un dossier `public` contenant un fichier `index.html`.

- **Le HTML** crée les boutons et les formulaires.
- **Le JavaScript (côté client)** utilise `fetch()`. C'est comme un coursier : il prend les données du formulaire, court vers le serveur (les routes API de l'étape 3), attend la réponse et revient mettre à jour la page sans que celle-ci ne se recharge.

Étape 5 : Comprendre l'ID (Le badge unique)

Vous avez remarqué que MongoDB génère un `_id` complexe (ex: `659af...`).

- **Explication pour les débutants** : Imaginez une ville où deux personnes s'appellent "Jean Dupont". Pour la mairie, c'est l'enfer. MongoDB donne donc un "numéro de sécurité sociale" unique et impossible à deviner à chaque donnée. C'est grâce à ce badge qu'on peut dire au serveur : "Supprime *ce* Jean Dupont là, et pas l'autre".

Résumé du flux d'une action

Quand vous cliquez sur "Enregistrer" :

1. **Navigateur** : "Hé JavaScript, prends ces infos et envoie-les à `/users` en mode POST."
2. **Serveur (Node)** : "Je reçois un POST. Mongoose, vérifie si c'est conforme au moule (Schema)."
3. **Base de données (MongoDB)** : "C'est bon, je l'ai rangé et je lui ai donné l'ID `xyz`."
4. **Serveur** : "Ok, je renvoie le nouvel utilisateur au navigateur."
5. **Navigateur** : "Super, je l'ajoute à la liste affichée à l'écran."

Un conseil de formateur :

Ne vous inquiétez pas si vous ne comprenez pas chaque ligne de code du premier coup. L'important est de comprendre **le trajet de la donnée** : du formulaire vers la base de données, et de la base de données vers l'écran.

Voici votre **guide de survie complet** pour monter votre premier projet Full-Stack de A à Z. Suivez ces étapes dans l'ordre, et vous aurez votre application fonctionnelle en moins de 10 minutes.

Étape 1 : Installation des outils (Le matériel)

Avant de coder, assurez-vous d'avoir installé ces deux logiciels sur votre ordinateur :

1. **Node.js** : Télécharger ici. Prenez la version "LTS".
2. **MongoDB Community Server** : Télécharger ici. Installez-le en laissant les options par défaut.

Étape 2 : Création du projet

1. Créez un dossier nommé `mon-projet-crud` sur votre bureau.
2. Ouvrez ce dossier dans **Visual Studio Code**.
3. Ouvrez le terminal de VS Code (`Ctrl + ù` ou `Terminal > Nouveau Terminal`) et tapez :
   Bash

   ```
   npm init -y
   ```

4. ```
   npm install express mongoose dotenv
   ```
5.

   Ceci va créer votre fichier `package.json` et télécharger vos outils.

Étape 3 : Création des fichiers

Créez exactement ces trois fichiers à la racine de votre dossier :

1. Fichier `.env` (Vos réglages)

Plaintext

```
MONGO_URI=mongodb://127.0.0.1:27017/bdd-node
PORT=3000
```
```

2. Fichier `server.js` (Votre serveur)

```
const express = require('express');
const mongoose = require('mongoose');
require('dotenv').config(); // Pour charger les variables
d'environnement


const app = express();
const PORT = process.env.PORT || 3000;

// Middleware pour parser le JSON
app.use(express.json());

// --- CONNEXION À MONGODB ---
// Remplacez MONGO_URI par votre chaîne de connexion dans un
fichier .env
const mongoURI = process.env.MONGO_URI;

if (!mongoURI) {
    console.error("ERREUR : La variable MONGO_URI n'est pas
définie dans le fichier .env");
    process.exit(1); // Arrête l'application car elle ne peut pas
fonctionner sans base de données
}

mongoose.connect(mongoURI)
    .then(() => {
        console.log('✅ Connexion à MongoDB réussie !');

        // On ne lance le serveur que si la connexion à la base
réussit
        app.listen(PORT, () => {
            console.log(`🚀 Serveur lancé sur http://localhost:$
{PORT}`);
        });
    })
    .catch((err) => {
        console.error('❌ Erreur de connexion à MongoDB :',
err.message);
    });

    const userSchema = new mongoose.Schema({
    nom: { type: String, required: true },
    email: { type: String, required: true, unique: true },
    age: Number,
    createdAt: { type: Date, default: Date.now }
    });

    const User = mongoose.model('User', userSchema);
```

```
// On défini les Routes
app.use(express.static('public'));

//Ajouter un utilisateur (CREATE)
app.post('/users', async (req, res) => {
    try {
        const newUser = new User(req.body);
        await newUser.save();
        res.status(201).json(newUser);
    } catch (err) {
        res.status(400).json({ error: err.message });
    }
});
//Lire tous les utilisateurs (READ)
app.get('/users', async (req, res) => {
    try {
        const users = await User.find();
        res.json(users);
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
});
//Modifier un utilisateur (UPDATE)
//On utilise l'ID généré par MongoDB pour cibler l'utilisateur.
app.put('/users/:id', async (req, res) => {
    try {
        const updatedUser = await
User.findByIdAndUpdate(req.params.id, req.body, { new: true });
        res.json(updatedUser);
    } catch (err) {
        res.status(400).json({ error: err.message });
    }
});
//Supprimer un utilisateur (DELETE)
app.delete('/users/:id', async (req, res) => {
    try {
        await User.findByIdAndDelete(req.params.id);
        res.json({ message: "Utilisateur supprimé avec succès" });
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
});
```

3. Dossier `public` et fichier `index.html`

- Créez un dossier nommé `public`.
- À l'intérieur de `public`, créez `index.html`

```html
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>Gestion Utilisateurs - CRUD</title>
    <style>
        body { font-family: 'Segoe UI', Tahoma, Geneva, Verdana,
sans-serif; max-width: 700px; margin: 20px auto; background-color:
#f8f9fa; color: #333; }
        h2 { border-bottom: 2px solid #28a745; padding-bottom:
10px; }
        form { background: #ffffff; padding: 20px; border-radius:
8px; box-shadow: 0 2px 5px rgba(0,0,0,0.1); margin-bottom: 30px; }
        input { display: block; margin: 10px 0; width: 100%;
padding: 10px; border: 1px solid #ddd; border-radius: 4px; box-
sizing: border-box; }

        .btn-group { display: flex; gap: 10px; }
        button { cursor: pointer; padding: 10px 15px; border:
none; border-radius: 4px; font-weight: bold; transition: opacity
0.2s; }
        button:hover { opacity: 0.8; }

        .btn-save { background: #28a745; color: white; width:
100%; }
        .btn-edit { background: #007bff; color: white; padding:
5px 10px; }
        .btn-delete { background: #dc3545; color: white; padding:
5px 10px; }
        .btn-cancel { background: #6c757d; color: white; margin-
top: 5px; width: 100%; display: none; }

        .user-card { background: white; border: 1px solid #eee;
padding: 15px; margin-bottom: 10px; border-radius: 6px; display:
flex; justify-content: space-between; align-items: center; box-
shadow: 0 1px 3px rgba(0,0,0,0.05); }
        .user-info { display: flex; flex-direction: column; }
        .user-actions { display: flex; gap: 5px; }
    </style>
</head>
<body>

    <h2>👤 Gestion des Utilisateurs</h2>

    <form id="userForm">
        <h3 id="formTitle">Ajouter un utilisateur</h3>
        <input type="text" id="nom" placeholder="Nom complet"
required>
```

```
        <input type="email" id="email" placeholder="Adresse email"
required>
        <input type="number" id="age" placeholder="Âge">
        <button type="submit" id="submitBtn" class="btn-
save">Enregistrer</button>
        <button type="button" id="cancelBtn" class="btn-cancel"
onclick="resetForm()">Annuler la modification</button>
    </form>

    <h2>📋 Liste des inscrits</h2>
    <div id="userList">
        </div>

    <script>
        const userForm = document.getElementById('userForm');
        const userList = document.getElementById('userList');
        const submitBtn = document.getElementById('submitBtn');
        const cancelBtn = document.getElementById('cancelBtn');
        const formTitle = document.getElementById('formTitle');

        let editId = null; // Stocke l'ID de l'utilisateur en
cours de modification

        // 1. Charger et afficher les utilisateurs
        async function loadUsers() {
            try {
                const res = await fetch('/users');
                const users = await res.json();

                if (users.length === 0) {
                    userList.innerHTML = "<p>Aucun utilisateur
trouvé.</p>";
                    return;
                }

                userList.innerHTML = users.map(user => `
                    <div class="user-card">
                        <div class="user-info">
                            <strong>${user.nom}</strong>
                            <span>${user.email} • ${user.age ||
'?'} ans</span>
                        </div>
                        <div class="user-actions">
                            <button class="btn-edit"
onclick="prepareEdit('${user._id}', '${user.nom}', '$
{user.email}', ${user.age})">Modifier</button>
                            <button class="btn-delete"
onclick="deleteUser('${user._id}')">Supprimer</button>
                        </div>
                    </div>
                `).join('');
```

```
            } catch (err) {
                console.error("Erreur chargement:", err);
            }
        }

        // 2. Ajouter ou Modifier un utilisateur
        userForm.onsubmit = async (e) => {
            e.preventDefault();

            const data = {
                nom: document.getElementById('nom').value,
                email: document.getElementById('email').value,
                age: document.getElementById('age').value
            };

            const url = editId ? `/users/${editId}` : '/users';
            const method = editId ? 'PUT' : 'POST';

            try {
                const response = await fetch(url, {
                    method: method,
                    headers: { 'Content-Type': 'application/
json' },
                    body: JSON.stringify(data)
                });

                if (!response.ok) {
                    const errorData = await response.json();
                    alert("Erreur: " + errorData.error);
                } else {
                    resetForm();
                    loadUsers();
                }
            } catch (err) {
                alert("Erreur de connexion au serveur");
            }
        };

        // 3. Préparer la modification (remplir le formulaire)
        function prepareEdit(id, nom, email, age) {
            editId = id;
            document.getElementById('nom').value = nom;
            document.getElementById('email').value = email;
            document.getElementById('age').value = age ===
'null' ? '' : age;

            formTitle.innerText = "Modifier l'utilisateur";
            submitBtn.innerText = "Mettre à jour";
            submitBtn.style.background = "#007bff";
            cancelBtn.style.display = "block";
            window.scrollTo(0, 0); // Remonte au formulaire
        }
```

```
        // 4. Supprimer un utilisateur
        async function deleteUser(id) {
            if (confirm('Voulez-vous vraiment supprimer cet
utilisateur ?')) {
                await fetch(`/users/${id}`, { method: 'DELETE' });
                loadUsers();
            }
        }

        // 5. Réinitialiser le formulaire
        function resetForm() {
            editId = null;
            userForm.reset();
            formTitle.innerText = "Ajouter un utilisateur";
            submitBtn.innerText = "Enregistrer";
            submitBtn.style.background = "#28a745";
            cancelBtn.style.display = "none";
        }

        // Lancement initial
        loadUsers();
    </script>
</body>
</html>
```

Étape 4 : Lancement (Les commandes terminal)

C'est le moment de vérité ! Dans votre terminal VS Code, saisissez ces instructions :

1. **Démarrer le serveur :**
   Bash

   ```
   node server.js
   ```

2.

3. **Vérifier le message :** Vous devez voir s'afficher :
   - ✅ `Connexion à MongoDB réussie !`
   - 🚀 `Serveur lancé sur http://localhost:3000`

Étape 5 : Affichage dans le navigateur

1. Ouvrez votre navigateur (Chrome, Firefox ou Edge).

2. Dans la barre d'adresse, tapez : 👉 **`http://localhost:3000`**

3. Testez l'ajout d'un utilisateur. Si l'ID s'affiche dans la liste, félicitations : votre donnée a fait l'aller-retour jusqu'à la base de données !

Aide au débogage :

- **Erreur `ECONNREFUSED` ?** Votre logiciel MongoDB n'est pas démarré sur votre PC.
- **Page blanche ?** Vérifiez que votre fichier s'appelle bien `index.html` et qu'il est bien **dans** le dossier `public`.
- **Le site ne se met pas à jour ?** Quand vous modifiez `server.js`, vous devez arrêter le terminal (`Ctrl + C`) et relancer `node server.js`.

Si vous êtes sour MAC:

1. La "Connexion String" : La clé de l'armoire

Pour que Compass (ou n'importe quel logiciel) voit vos données, il doit se brancher sur la même adresse que votre fichier `.env`.

1. Ouvrez MongoDB Compass.
2. Cliquez sur "**New Connection**".
3. Dans le champ **URI**, copiez exactement la même adresse que dans votre projet : 👉 `mongodb://127.0.0.1:27017`
4. Cliquez sur **Connect**.

2. Où sont vos données ? (La hiérarchie)

Une fois connecté, vous allez voir une liste à gauche. C'est ici que les apprenants sont souvent perdus. Voici l'organisation :

- **Cluster/Serveur** (L'adresse IP) : C'est votre Mac.
- **Database** (La Base) : Elle doit s'appeler **`bdd-node`** (c'est le nom que nous avons mis à la fin de l'URL dans le `.env`).
- **Collection** : C'est comme un onglet Excel.
  Elle s'appellera **`users`**. Mongoose met automatiquement le nom de votre modèle au pluriel.

3. Comment l'utiliser en démonstration ?

Voici ce que vous pouvez montrer :

1. Ouvrez votre site `localhost:3000` d'un côté et Compass de l'autre.
2. Ajoutez un utilisateur sur le site.
3. Cliquez sur le bouton "**Refresh**" (actualiser) dans Compass.
4. **Magie :** Le document apparaît instantanément sous forme de JSON avec son fameux `_id` !

4. Attention au piège sur MacBook (M1/M2/M3/M4/M5)

Si vous avez l'erreur "**Connection Refused**" dans Compass :

- C'est que le **Service MongoDB** n'est pas lancé en arrière-plan sur votre Mac.
- Solution rapide dans votre terminal Mac :
  Bash

  ```
  brew services start mongodb-community
  ```

-

Le conseil :

Expliquez bien que **Compass est un "spectateur"**.
On peut modifier les données directement dedans (double-clic sur un nom), et si on rafraîchit le site web, le nom aura changé.
Cela prouve que le site et Compass regardent **la même source de vérité**.

Est-ce que vous arrivez à voir votre base `bdd-node` s'afficher dans la colonne de gauche de l'application ?